



BRAIN TUMOR MRI CLASSIFICATION

A Comprehensive Machine Learning Study:
Classical ML, Deep Learning & Transfer Learning

Table of Contents

1. INTRODUCTION.....	3
1.1. Problem Statement	3
1.2. Classification Task	3
1.3. Project Objectives	3
1.4. Python Libraries	4
2. DATA EXPLORATION	5
2.1. Data set: Dataset_MRI	5
2.2. Class Distribution	5
2.3. Train and Test data (Images) Visualization	6
3. CLASSICAL MACHINE LEARNING APPROACHES (PART-1).....	7
3.1. Feature Extraction.....	7
3.2. Data Preprocessing Pipeline	7
3.3. Classical ML Models (KNN, SVM, Random Forest)	8
3.3.1. Model (Default Parameters)	8
3.3.2. Model (Hyperparameters Parameters)	9
3.3.3. Learning Curves	10
3.4. Key Findings from Classical Machine Learning (Part 1)	11
4. CONVOLUTIONAL NEURAL NETWORKS WITH GRAD-CAM (PART2)	12
4.1. Convolutional Neural Networks (CNNs)	12
4.1.1. CNN Architecture Principles.....	12
4.1.2. CNN Architecture	12
4.1.3. Accuracy and Improvements:	14
4.2. Grad-CAM: Visual Explainability	14
4.2.1. What is Grad-CAM?	14
4.2.2. Key Advantages of Grad-CAM	15
4.3. Key Findings from Part 2	15
5. TRANSFER LEARNING WITH RESNET50 AND LIME (PART-3).....	17
5.1. Transfer Learning	17
5.2. ResNet50 Architecture.....	17
5.3. Training and Testing	18
5.4. LIME: Local Interpretable Model-agnostic Explanations	19
5.4.1. What is LIME?	19
5.4.2. LIME vs Grad-CAM Comparison.....	21
5.5. Key Findings from Part 3	21

6.	OVERALL CONCLUSION AND COMPARATIVE ANALYSIS	22
6.1.	Performance Comparison	22
6.2.	Method Characteristics.....	22
7.	PROJECT SUMMARY.....	23

1. INTRODUCTION

This comprehensive report presents three progressive machine learning approaches for brain tumor classification from MRI images.

- Classical Machine Learning (PART 1)
- Convolutional Neural Networks with Grad-CAM (PART 2)
- Transfer Learning with ResNet50 and LIME (PART 3)

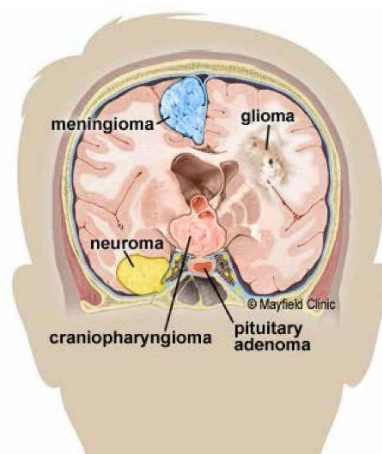
1.1. Problem Statement

Brain tumors represent a critical medical challenge requiring accurate classification for effective treatment planning. This project develops and compares multiple machine learning pipelines for multiclass image classification of brain tumors in MRI scans, categorizing them into four tumor types: Glioma, Meningioma, No Tumor, and Pituitary tumors.

1.2. Classification Task

This is a 4-class image classification problem with the following tumor types:

Class	Description
Glioma	Tumor originating from the brain's glial cells
Meningioma	Tumor in the protective membranes surrounding the brain
No Tumor	MRI scans with no detectable brain tumor
Pituitary	Tumor in the pituitary gland at the brain's base



1.3. Project Objectives

- Build and compare multiple machine learning pipelines for brain tumor classification
- Implement both classical and deep learning approaches
- Develop explainable AI models suitable for medical diagnosis
- Evaluate and compare model performance across different architectures

- Provide insights into which image regions are important for tumor classification

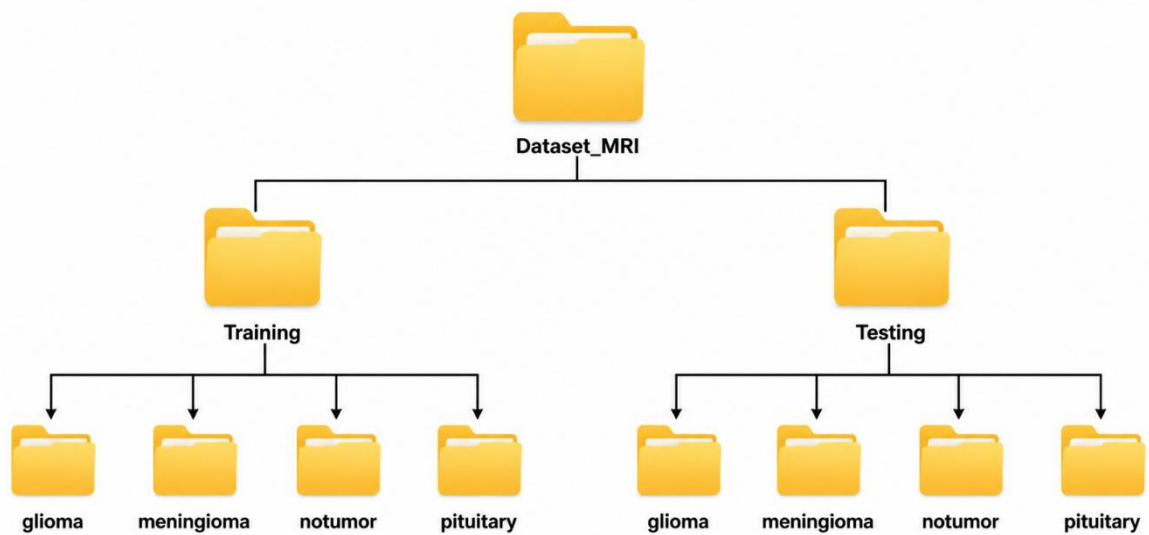
1.4. Python Libraries

python version ≤ 3.12 for Tensor flow

- numpy
- pandas
- matplotlib
- seaborn
- plotly
- opencv-python
- scikit-image
- scikit-learn
- tensorflow
- lime
- tqdm
- imutils

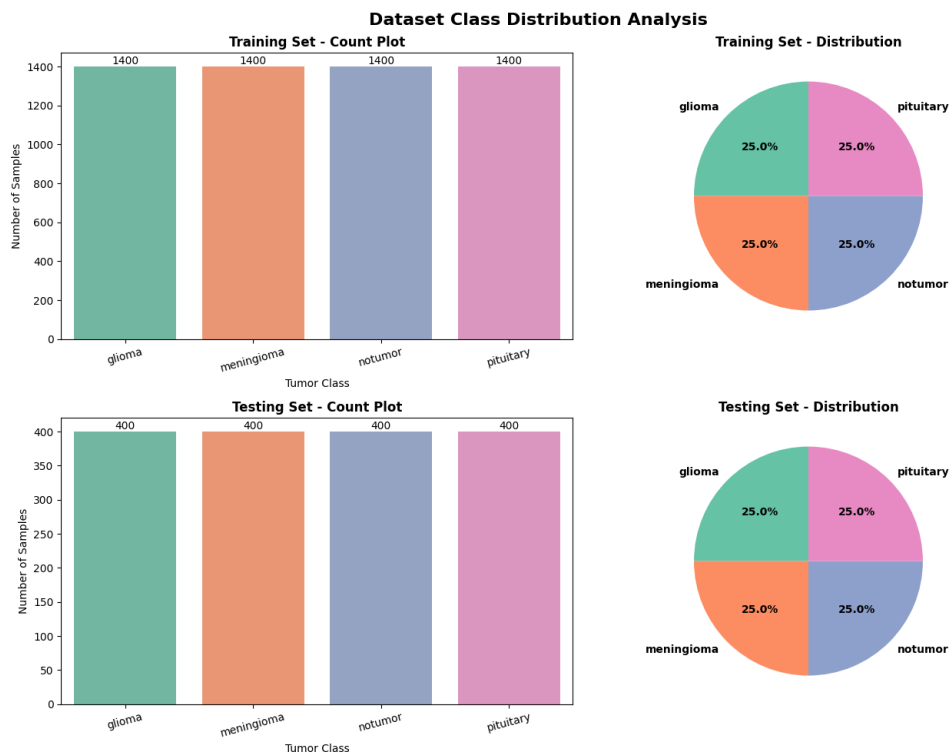
2. DATA EXPLORATION

2.1. Data set: Dataset_MRI



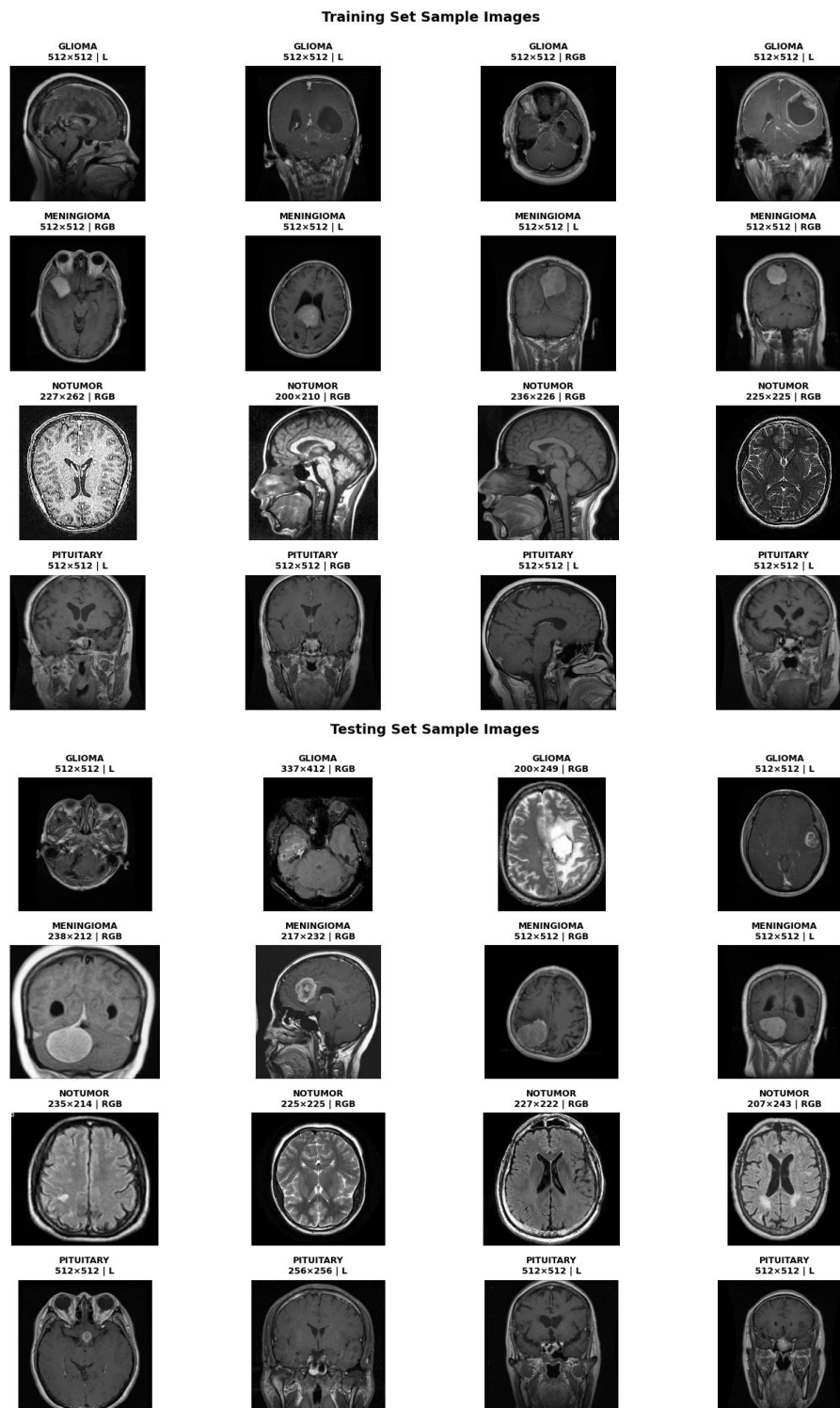
2.2. Class Distribution

Dataset is well-balanced across all classes



2.3. Train and Test data (Images) Visualization

- Images have different sizes (not uniform), All images will be resized to 128×128 for consistency
- Images have different color modes (grayscale vs RGB), Pipeline handles both by converting to RGB first then to grayscale for feature extraction



3. CLASSICAL MACHINE LEARNING APPROACHES (PART-1)

Part 1 implements classical machine learning pipelines using handcrafted features extracted from MRI images.

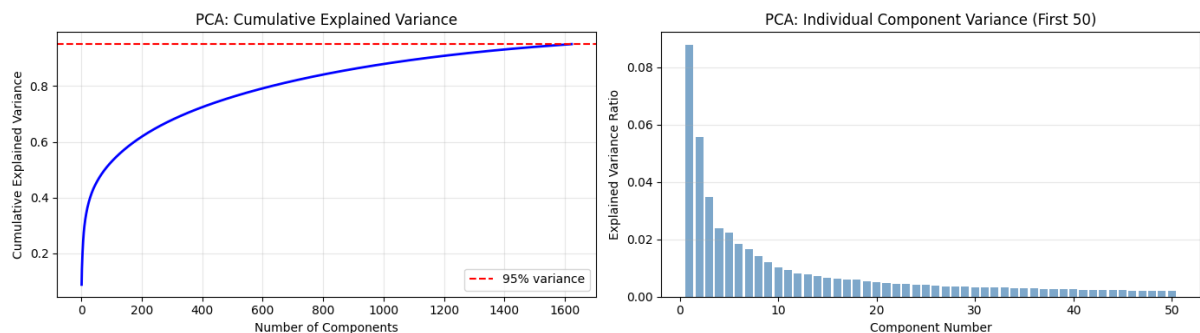
3.1. Feature Extraction

Two types of handcrafted features are extracted from MRI images:

- **Histogram of Oriented Gradients (HOG):**
 - Captures edge and shape information
 - Divides image into cells and computes gradient histograms
 - Parameters: 9 orientations, 8×8 pixels per cell
- **Gray Level Co-occurrence Matrix (GLCM) Texture Features:**
 - Measures spatial relationship of pixel intensities
 - Extracts texture properties: contrast, dissimilarity, homogeneity, energy, ASM
 - Captures fine-grained texture information

3.2. Data Preprocessing Pipeline

1. Load all MRI images from dataset
2. Convert images to RGB then grayscale
3. Convert all images to same Scale 128x128
4. Normalize pixel values to [0, 1]
5. Extract HOG features
6. Extract GLCM texture features
7. Concatenate feature vectors
8. Apply StandardScaler normalization
9. Apply PCA for dimensionality reduction (95% variance)
 - i. Original dimensions: 8,106
 - ii. Reduced dimensions: 1,622
 - iii. Variance retained: 95.00%
 - iv. Reduction ratio: 5.0×



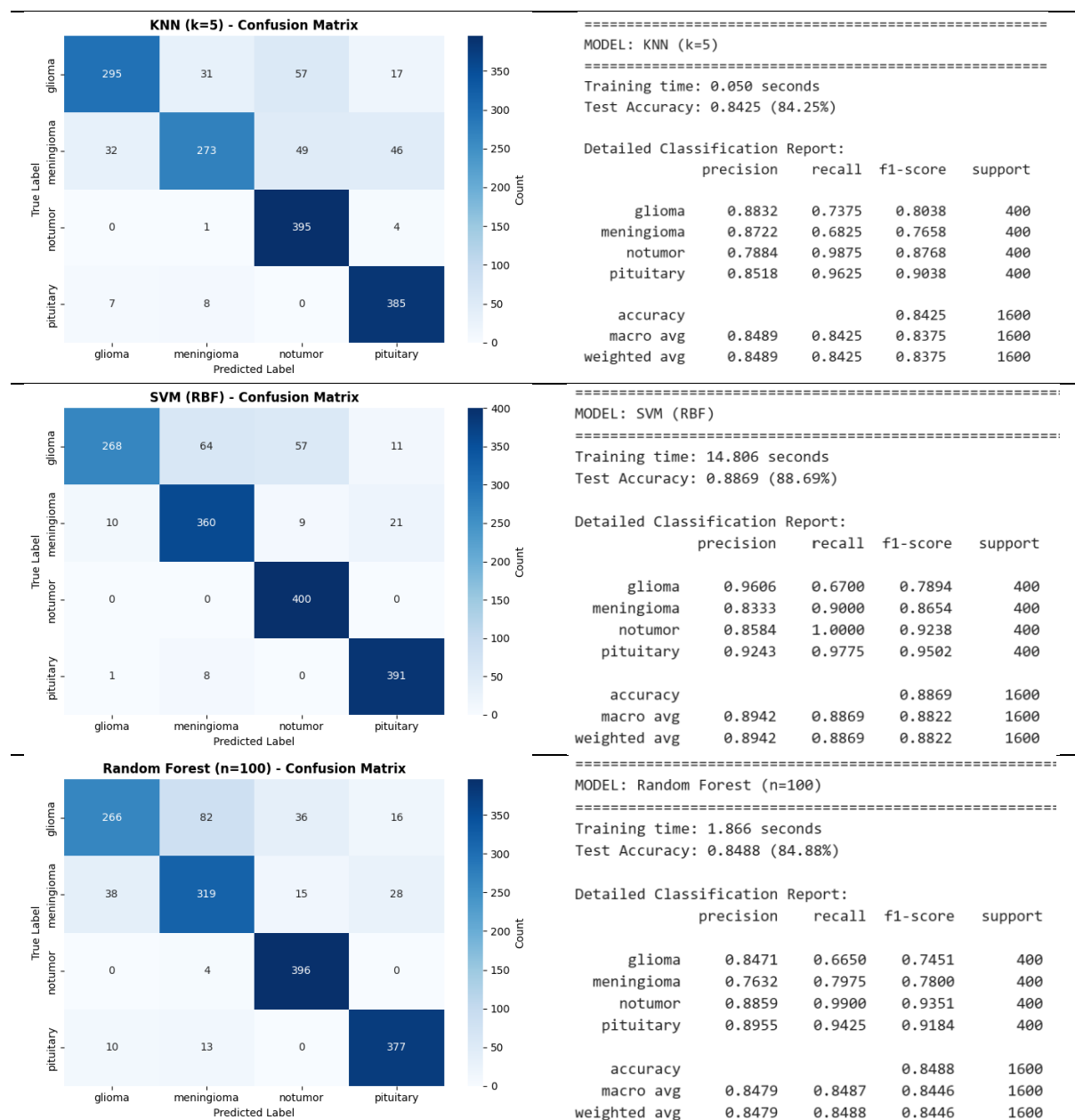
3.3. Classical ML Models (KNN, SVM, Random Forest)

- K-Nearest Neighbours (KNN): Distance-based (proximity)
- Support Vector Machine (SVM): Boundary-based (hyperplanes)
- Random Forest: Ensemble-based (multiple decision trees)

3.3.1. Model (Default Parameters)

Train baseline models KNN ($k = 5$), SVM ($kernel = rbf$), Random Forest ($n = 100$) with default hyperparameters to establish performance baseline.

- Results are:



SVM Performs better

3.3.2. Model (Hyperparameters Parameters)

- 5-Fold Cross-Validation (CV) with GridSearchCV

- *KNN GridSearch Parameters*

Hyperparameter	Values to Test
n_neighbors (k)	[3, 5, 7, 9, 11]
weights	['uniform', 'distance']
metric	['euclidean', 'manhattan', 'minkowski']
TOTAL COMBINATIONS	$5 \times 2 \times 3 = 30$

- *SVM GridSearch Parameters*

Hyperparameter	Values to Test
C (Regularization)	[0.1, 1, 10, 100]
kernel	['linear', 'rbf', 'poly']
gamma	['scale', 'auto', 0.001, 0.01, 0.1]
TOTAL COMBINATIONS	$4 \times 3 \times 5 = 60$

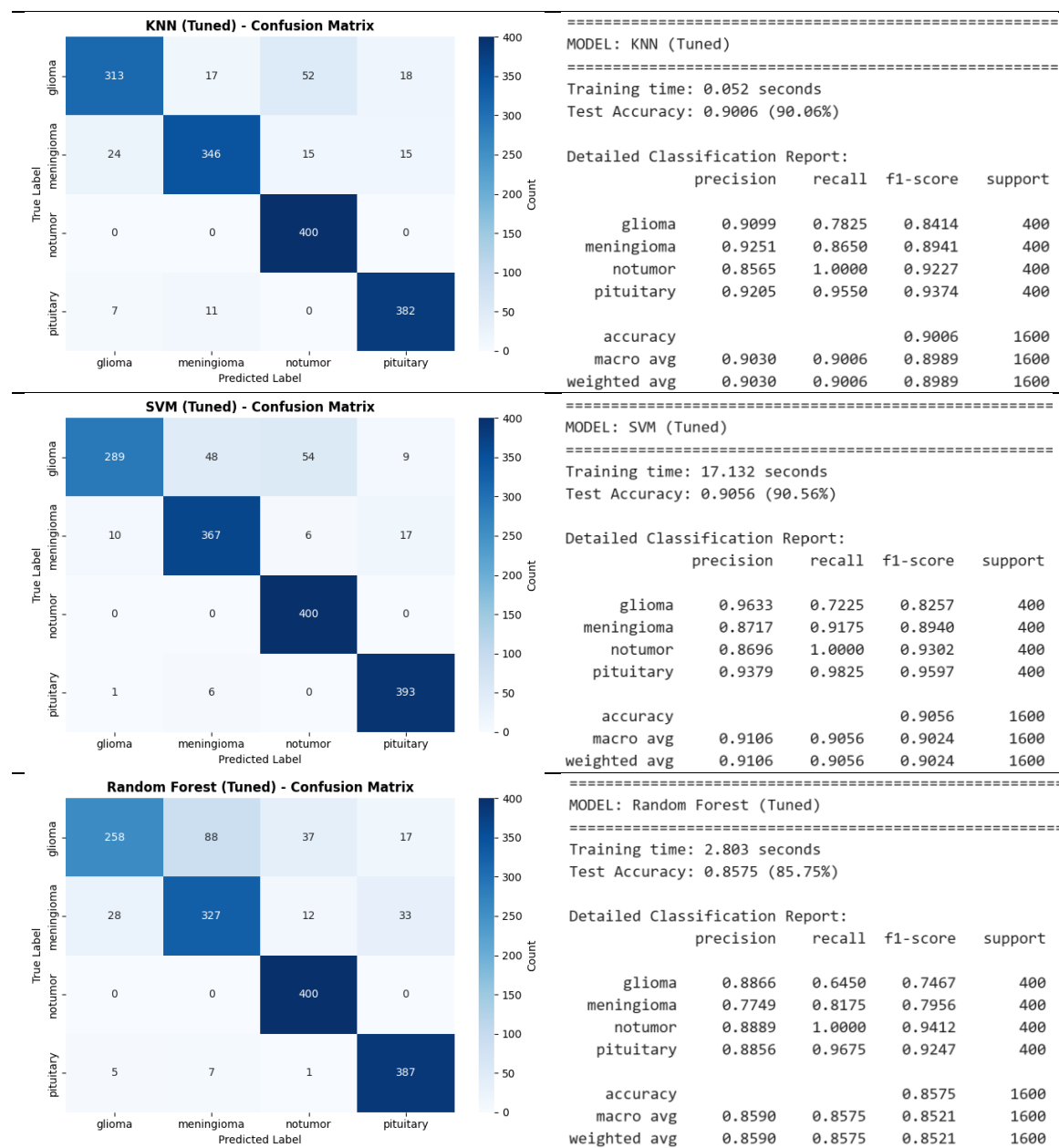
- *Random Forest GridSearch Parameters*

Hyperparameter	Values to Test
n_estimators	[50, 100, 150]
max_depth	[None, 10, 20, 30]
min_samples_split	[2, 5, 10]
min_samples_leaf	[1, 2, 4]
max_features	['sqrt', 'log2']
TOTAL COMBINATIONS	$3 \times 4 \times 3 \times 3 \times 3 = 216$

- Best Parameters and CV score

Classifier	Best Parameters	Best CV Score:
KNN	{'metric': 'euclidean', 'n_neighbors': 3, 'weights': 'distance'}	0.9393
SVM	{'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}	0.9461
Random Forest	{'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 10, 'n_estimators': 150}	0.8816

- Training with Best Parameters



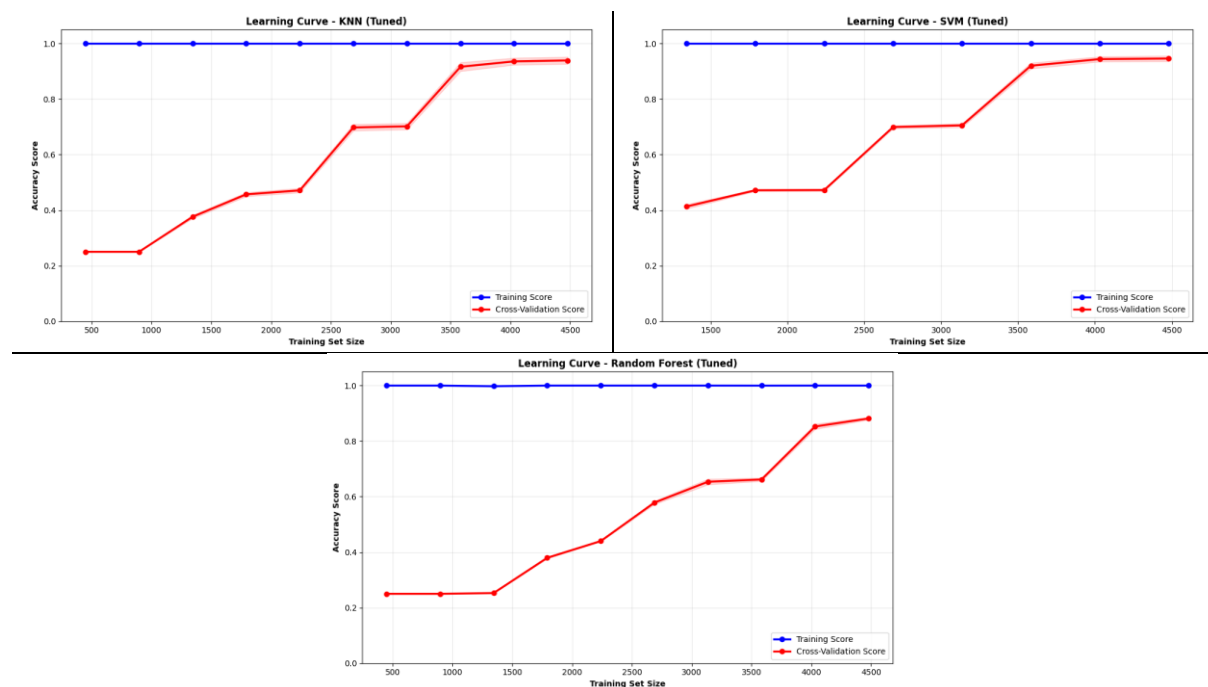
SVM Performs better with 90.56% accuracy

3.3.3. Learning Curves

As the training size increases, the training accuracy stays close to 1.0 for all three models, which indicates that each tuned model fits the training data almost perfectly. At the same time, the cross-validation accuracy steadily increases with more data, showing that the models generalize better when they see larger training sets. The gap between training and validation performance becomes smaller for larger sample sizes, suggesting that adding data reduces overfitting and stabilizes the models' performance.

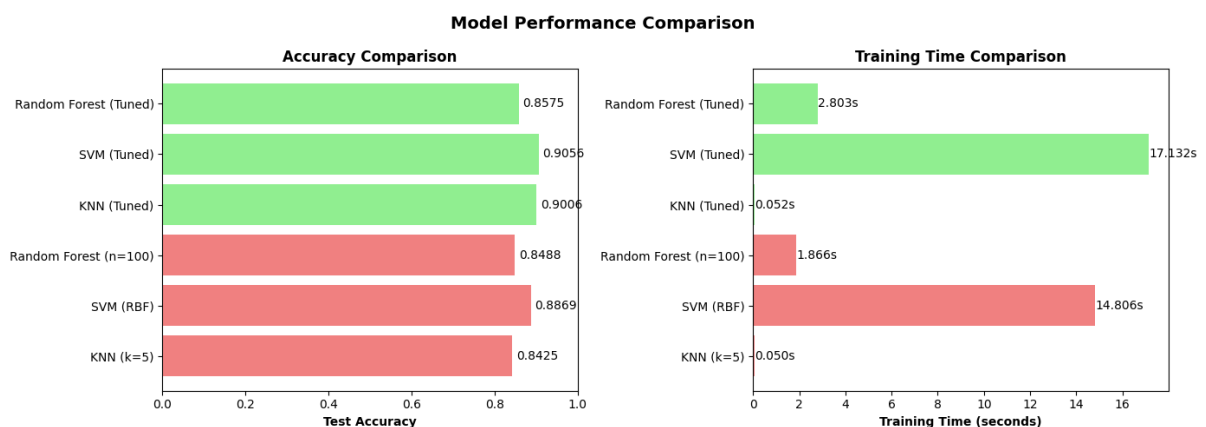
Among the three algorithms, all reach high validation accuracy when the training size is large. Any remaining gap between the blue and red curves indicates that the model is still somewhat overfitted,

whereas nearly overlapping curves would indicate that the model has learned a pattern that generalizes well to unseen data.



3.4. Key Findings from Classical Machine Learning (Part 1)

- Classical ML approaches rely heavily on feature engineering quality
- Combination of HOG and GLCM features captures both shape and texture information
- SVM outperform others
- Learning curves reveal potential for improvement with more training data (data augmentation using Albumentations (<https://albumentations.ai/docs/>) library)



4. CONVOLUTIONAL NEURAL NETWORKS WITH GRAD-CAM (PART2)

Part 2 implements a Convolutional Neural Network (CNN) for brain tumor classification with Grad-CAM visualization.

4.1. Convolutional Neural Networks (CNNs)

4.1.1. CNN Architecture Principles

CNNs are specifically designed for image processing:

Why:

- Learns hierarchical features from raw pixels (*No Feature Extraction Needed*)

How:

- Convolutional layers extract spatial features (edges → textures → patterns → objects)
- Pooling layers reduce dimensionality and increase invariance
- Fully connected layers perform final classification

4.1.2. CNN Architecture

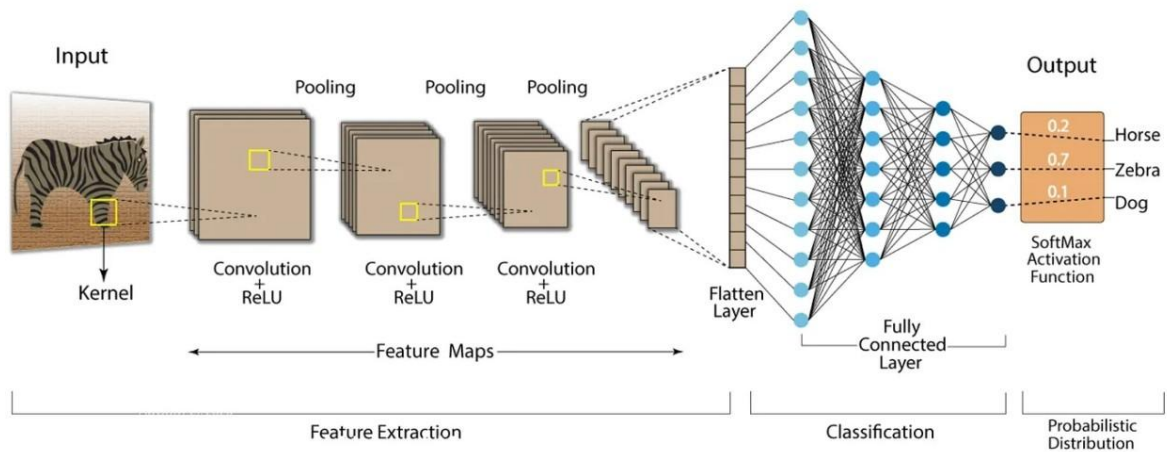
Configuration:

- Image Size: $224 \times 224 \times 3$ (RGB)
- Batch Size: 32
- Epochs: 15 (with callbacks: EarlyStopping, ReduceLROnPlateau, ModelCheckpoint)
- Validation Split: 0.3
- Learning Rate: 0.001
- Optimizer: Adam
- Loss: Categorical Cross-Entropy

Layers:

- Input layer: Takes the image into the model.
- Conv2D layers: Detect visual patterns like edges, shapes, and object parts.
- Batch Normalization: Makes training more stable and faster.
- MaxPooling2D: Reduces image size while keeping important features.
- Dropout: Helps prevent overfitting.
- GlobalAveragePooling2D: Compresses each feature map into one value.
- Dense layer: Combines features to make the final decision.
- Output layer: Gives the final class prediction.

Convolution Neural Network (CNN)



Model: "CNN_Network"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
conv1_1 (Conv2D)	(None, 224, 224, 32)	896
bn1_1 (BatchNormalization)	(None, 224, 224, 32)	128
conv1_2 (Conv2D)	(None, 224, 224, 32)	9,248
bn1_2 (BatchNormalization)	(None, 224, 224, 32)	128
pool1 (MaxPooling2D)	(None, 112, 112, 32)	0
dropout1 (Dropout)	(None, 112, 112, 32)	0
conv2_1 (Conv2D)	(None, 112, 112, 64)	18,496
bn2_1 (BatchNormalization)	(None, 112, 112, 64)	256
conv2_2 (Conv2D)	(None, 112, 112, 64)	36,928
bn2_2 (BatchNormalization)	(None, 112, 112, 64)	256
pool2 (MaxPooling2D)	(None, 56, 56, 64)	0
dropout2 (Dropout)	(None, 56, 56, 64)	0
conv4_1 (Conv2D)	(None, 56, 56, 256)	147,712
bn4_1 (BatchNormalization)	(None, 56, 56, 256)	1,024
last_conv (Conv2D)	(None, 56, 56, 256)	590,080
bn4_2 (BatchNormalization)	(None, 56, 56, 256)	1,024
pool4 (MaxPooling2D)	(None, 28, 28, 256)	0
dropout4 (Dropout)	(None, 28, 28, 256)	0
global_avg_pool (GlobalAveragePooling2D)	(None, 256)	0
dense1 (Dense)	(None, 256)	65,792
dropout5 (Dropout)	(None, 256)	0
output_layer (Dense)	(None, 4)	1,028

Total params: 872,996 (3.33 MB)

Trainable params: 871,588 (3.32 MB)

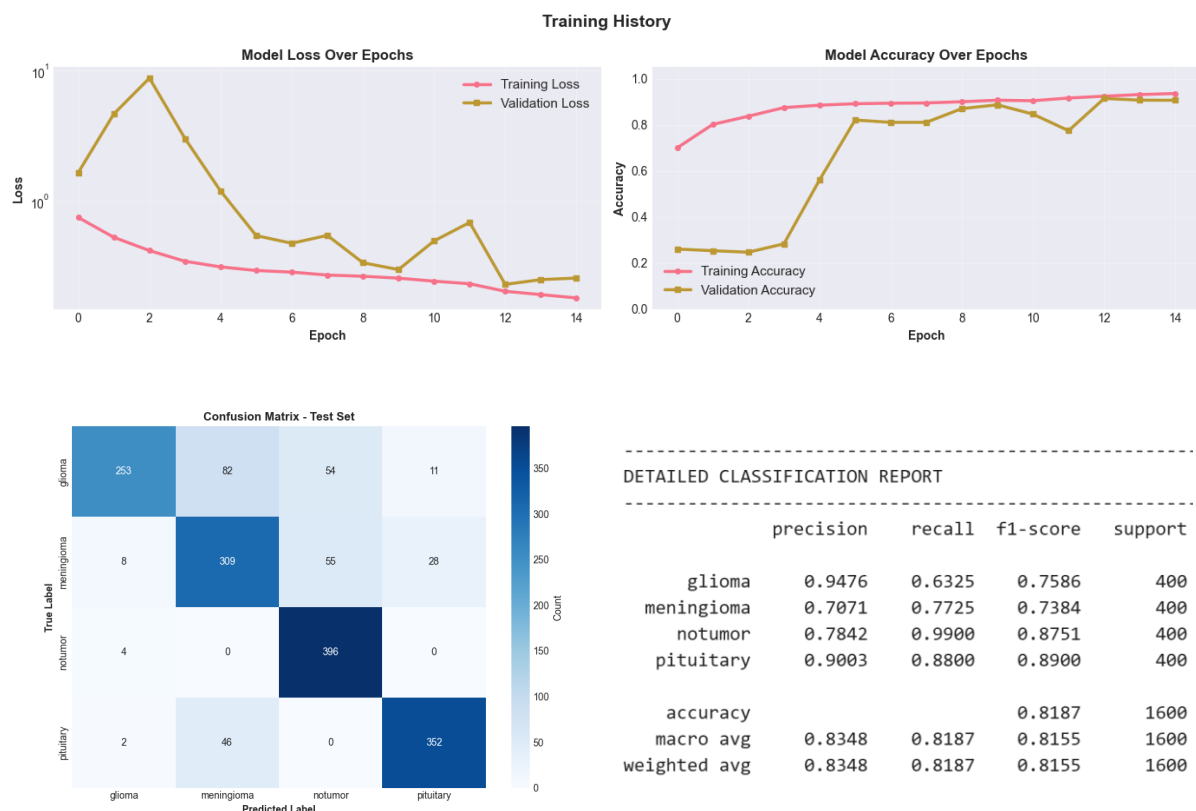
Non-trainable params: 1,408 (5.50 KB)

4.1.3. Accuracy and Improvements:

This training history looks good overall: training loss keeps going down, training accuracy keeps going up, and validation accuracy rises to around 0.9, so the model is learning well.

Test accuracy reached 81.87%. Further improvements may be achieved by

- Crop MRI images to remove background and focus on brain regions.
- Apply stronger data augmentation (rotations, shifts, zoom, brightness, flips) to improve robustness and reduce overfitting.
- Increase convolutional layers cautiously to capture complex features while controlling overfitting and computational cost.
- Use transfer learning (e.g., ResNet, EfficientNet) to improve performance (Part-3).



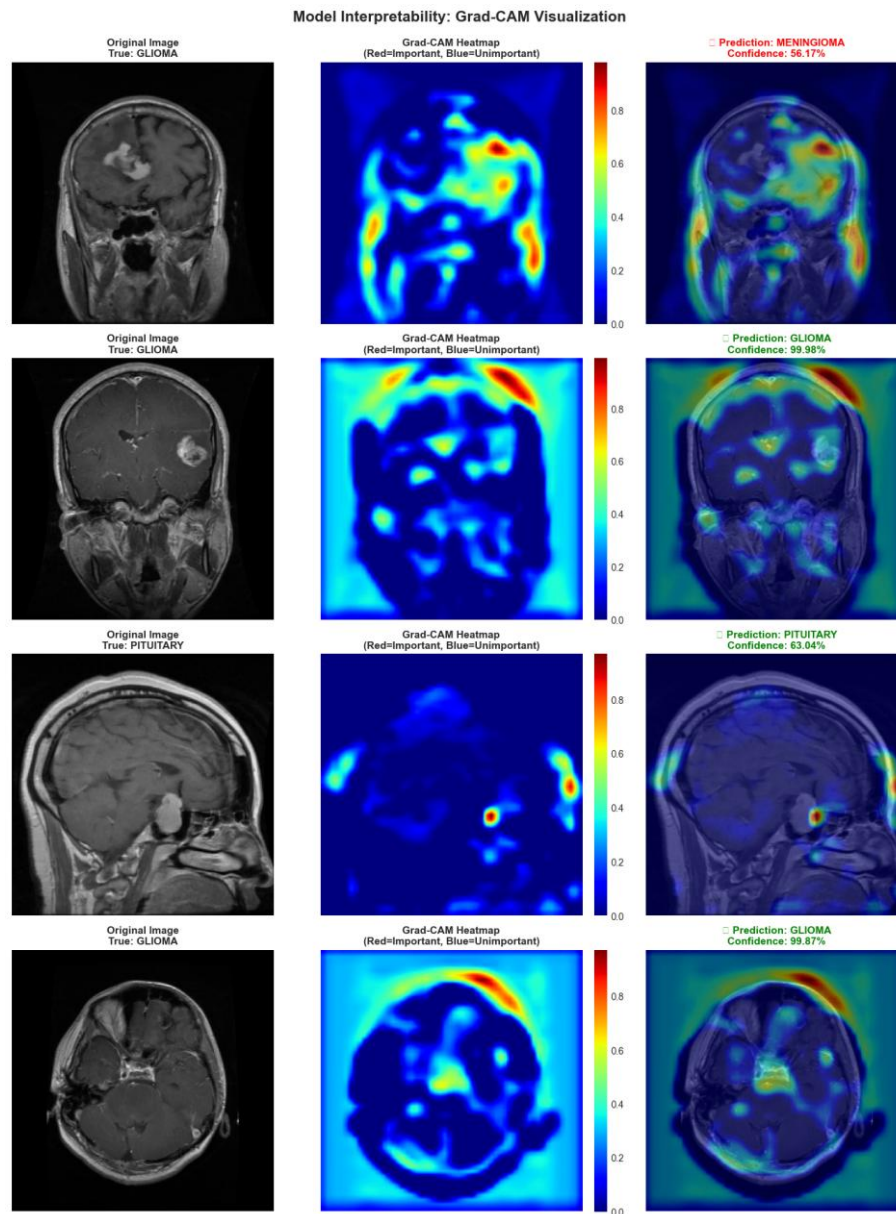
4.2. Grad-CAM: Visual Explainability

Grad-CAM (Gradient-weighted Class Activation Maps) is an explainability technique that shows which regions of an image influenced the model's prediction.

4.2.1. What is Grad-CAM?

- It creates a heatmap showing which parts of an image influenced a CNN's prediction
- Uses gradients of the target class flowing into the last convolutional layer
- Computes importance weights for each feature map by global average pooling the gradients
- Takes a weighted sum of all feature maps using these weights
- Applies ReLU to keep only features that positively affect the class
- Upsamples the result to input image size and overlays as a colored heatmap

- Works with any CNN architecture without retraining or structural changes
- Red regions = important for prediction, Blue regions = unimportant



4.2.2. Key Advantages of Grad-CAM

- Fast computation using gradient information
- Requires no model modification or retraining
- Provides class-specific explanations
- Ideal for medical imaging applications
- Can identify tumor regions in MRI scans

4.3. Key Findings from Part 2

- CNNs automatically learn hierarchical features without manual engineering
- Typical Accuracy: 81.87%.
- Grad-CAM provides valuable interpretability for medical applications

- Red regions align with actual tumor locations in many cases
- CNN captures spatial patterns that handcrafted features may miss
- Model explainability is crucial for clinical diagnosis systems

5. TRANSFER LEARNING WITH RESNET50 AND LIME (PART-3)

Part 3 applies transfer learning using pre-trained ResNet50 from ImageNet, combined with LIME for model interpretability.

5.1. Transfer Learning

Transfer learning leverages a model pre-trained on a large dataset (RESNET50) to improve performance on a smaller target dataset. This works because:

- Early layers learn general features (edges, textures) applicable to any image
- Later layers learn task-specific features (objects, patterns)
- We can reuse early/middle layers and only train task-specific layers

Benefits:

- Faster training (fewer parameters to train)
- Better generalization (leverages knowledge from large dataset)
- Requires less data (ImageNet knowledge compensates for limited medical images)

Aspect	Training from Scratch	Transfer Learning
Data Required	Large (100K+ images)	Small-Medium (1K-10K)
Training Time	Days/Weeks	Hours
Final Accuracy	Baseline	5-15% Higher

5.2. ResNet50 Architecture

- 50 Layers: Counts only layers with trainable weights (49 Convolutional + 1 Fully Connected).
- 175 Layers: Counts every single operational step executed in software frameworks.
 - The 175-Layer Breakdown:
 - 53 Convolutions (49 core + 4 shortcut adjustments)
 - 53 Batch Normalizations
 - 49 ReLU Activations
 - 16 Shortcut Additions
 - 4 Pooling/Output steps

50 indicates the weight layers; 175 indicates the total code operations.

5.3. Training and Testing

Frozen the base model, removed the final classification layer (added custom layer)

conv5_block3_out (Activation)	(None, 7, 7, 2048)	0	conv5_block3_add...
global_avg_pool (GlobalAveragePool...	(None, 2048)	0	conv5_block3_out...
dense1 (Dense)	(None, 512)	1,049,088	global_avg_pool[...
bn1 (BatchNormalizatio...	(None, 512)	2,048	dense1[0][0]
dropout1 (Dropout)	(None, 512)	0	bn1[0][0]
dense2 (Dense)	(None, 256)	131,328	dropout1[0][0]
bn2 (BatchNormalizatio...	(None, 256)	1,024	dense2[0][0]
dropout2 (Dropout)	(None, 256)	0	bn2[0][0]
output (Dense)	(None, 4)	1,028	dropout2[0][0]

Total params: 24,772,228 (94.50 MB)

Trainable params: 1,182,980 (4.51 MB)

Non-trainable params: 23,589,248 (89.99 MB)

Configuration:

- Image Size: $224 \times 224 \times 3$ (RGB) *must be* 224x224x3 for resnet50
- Batch Size: 32
- Epochs: 20 (with callbacks: EarlyStopping, ReduceLROnPlateau, ModelCheckpoint)
- Validation Split: 0.2
- Learning Rate: 0.001
- Optimizer: Adam
- Loss: Categorical Cross-Entropy

Accuracy: Overall Test Accuracy: 0.8981 (89.81%)

```
=====
TRAINING COMPLETED
=====
```

Training Statistics:

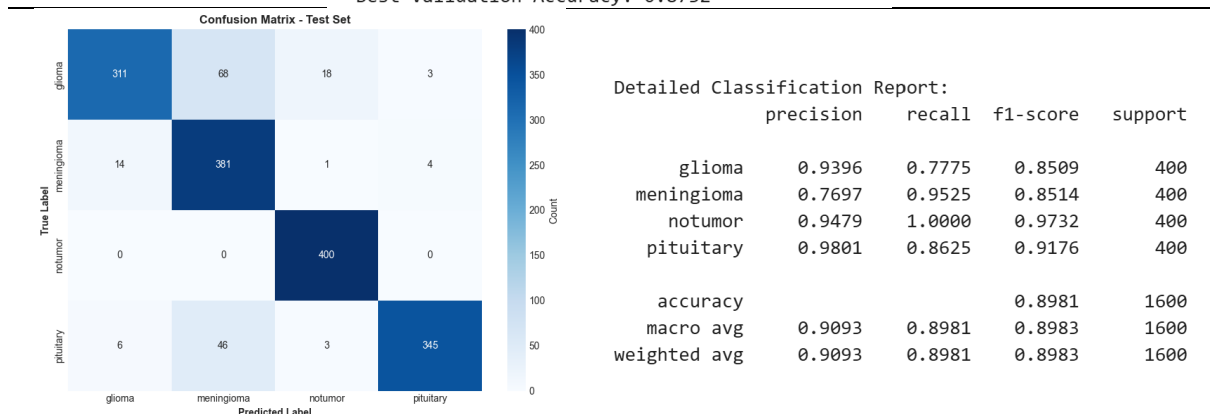
Actual epochs: 12 (stopped by EarlyStopping)
Training time: 2470.82 seconds (41.18 minutes)

Final Metrics:

Training Loss: 0.0569
Training Accuracy: 0.9833
Validation Loss: 0.4119
Validation Accuracy: 0.8679

Best Metrics:

Best Validation Loss: 0.4103
Best Validation Accuracy: 0.8732



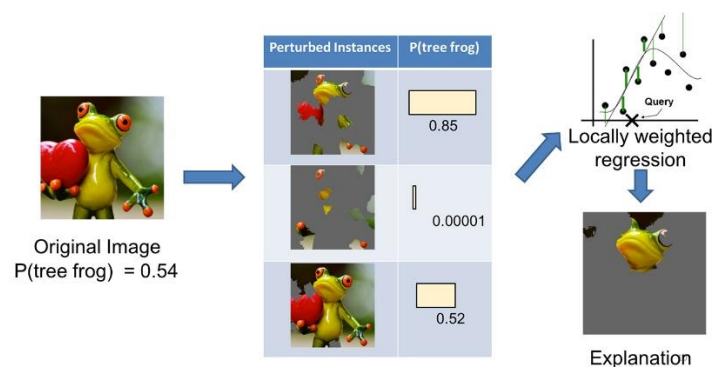
5.4. LIME: Local Interpretable Model-agnostic Explanations

5.4.1. What is LIME?

LIME is a technique to:

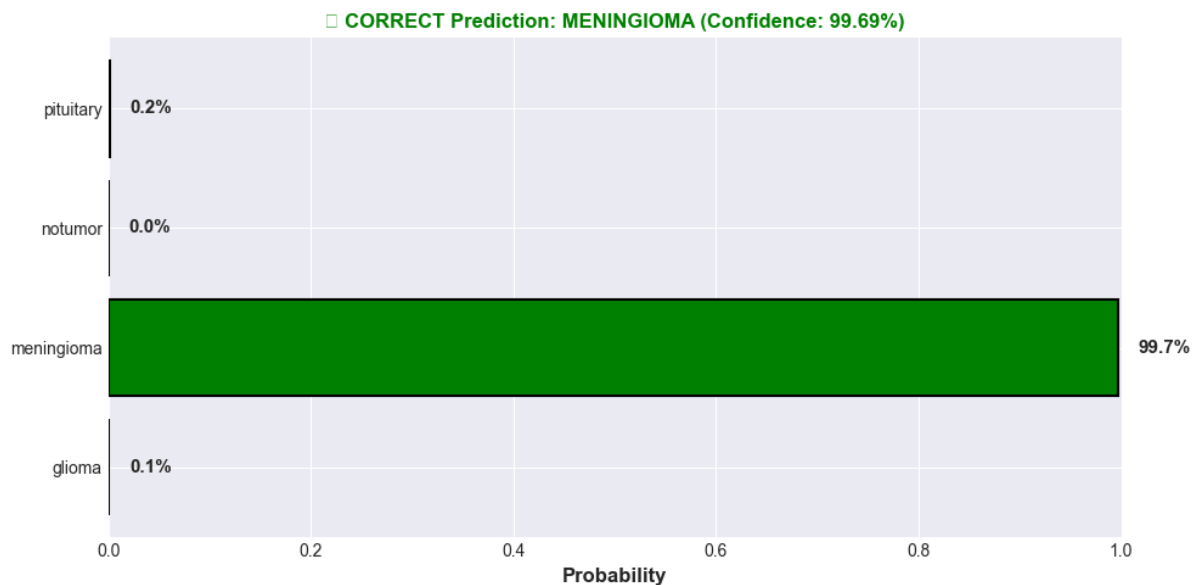
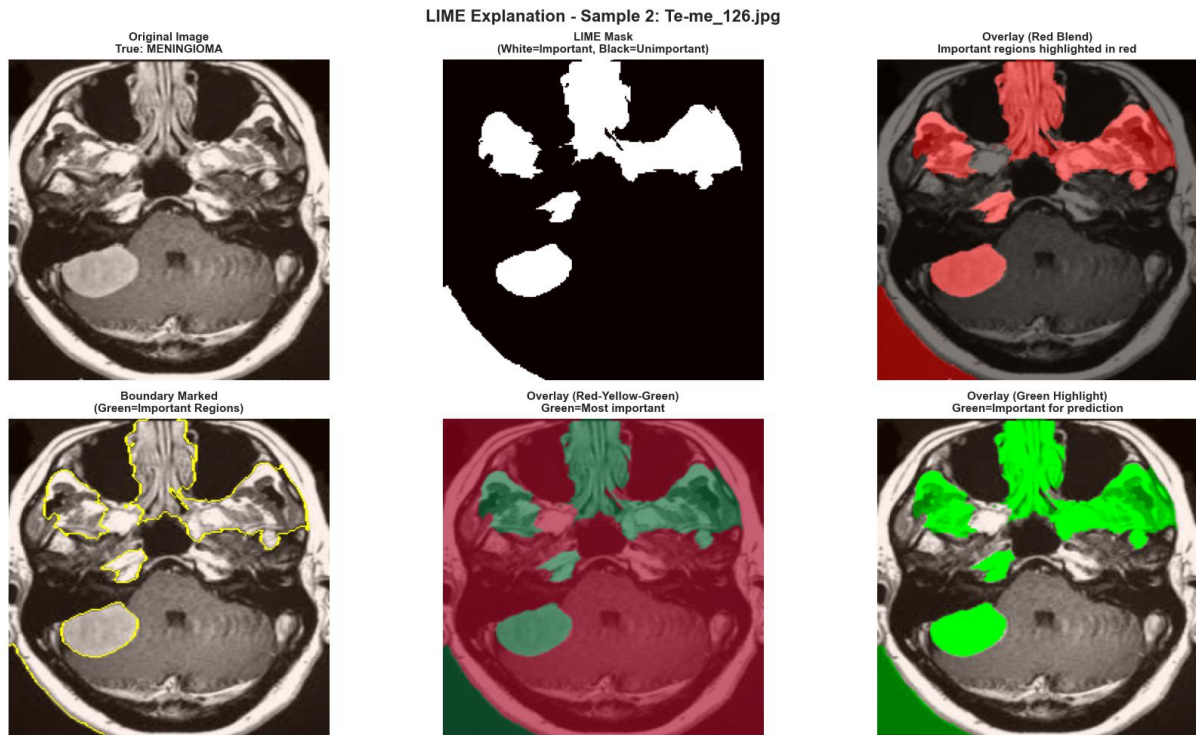
- Explain individual predictions
- Work with any model (model-agnostic) unlike GRAD CAM
- Approximate complex models locally with interpretable ones
- Show which features/super pixels influence specific predictions

How LIME Works:



- SELECT a sample prediction to explain

- PERTURB the input (create variations by masking regions)
- GET PREDICTIONS for perturbed inputs
- LEARN interpretable model (which perturbed features affect prediction)
- VISUALIZE important regions



Why LIME Matters:

- Makes black-box neural networks interpretable
- Shows which image regions influenced the decision
- Helps doctors understand model reasoning
- Critical for regulatory approval in healthcare

5.4.2. LIME vs Grad-CAM Comparison

Aspect	Grad-CAM	LIME
Model-specific	CNN only	Model-agnostic (Any Model)
Speed	Very Fast	Slow (many samples)
Output	Pixel heatmap	Superpixel importance

5.5. Key Findings from Part 3

- Transfer learning significantly improves performance with limited medical imaging data, Accuracy: 89.81% > CNN 81.87%, with less training time
- Pre-trained ImageNet features transfer well to brain tumor classification
- Fine-tuning all layers with very small learning rate achieves best results
- LIME provides model-agnostic explainability suitable for clinical settings
- Superpixel-level explanations are more granular than pixel-level heatmaps
- Combination of transfer learning and explainability enables trustworthy AI

6. OVERALL CONCLUSION AND COMPARATIVE ANALYSIS

6.1. Performance Comparison

Approach	Feature Type	Model	Accuracy
Part 1: Classical ML	HOG + GLCM	SVM / RF	90.56%
Part 2: CNN	Learned Features	Custom CNN	81.87%. %
Part 3: Transfer	ImageNet Features	ResNet50	89-90%

6.2. Method Characteristics

Criterion	Part 1: Classical ML	Part 2: CNN	Part 3: Transfer
Data Needed	Medium	Large	Small-Medium
Training Time	Fast	Slow	Fast
Interpretability	High	Medium	High

7. PROJECT SUMMARY

This comprehensive study demonstrates three progressive approaches to brain tumor classification from MRI images:

- PART 1 - Classical Machine Learning (90.56% Accuracy)
 - Focus: Handcrafted features (HOG + GLCM) with traditional classifiers (SVM, RF)
 - Benefit: Highly interpretable, fast training, good baseline
 - Challenge: Manual feature engineering required
- PART 2 - Convolutional Neural Networks (81.87% Accuracy)
 - Focus: Automatic feature learning with CNN + Grad-CAM visualization
 - Benefit: automatic feature discovery, visual explanations
 - Challenge: low accuracy, Requires more training data and computational resources
- PART 3 - Transfer Learning (89.81% Accuracy)
 - Focus: Pre-trained ResNet50 + LIME for model-agnostic explanations
 - Benefit: Best accuracy, efficient training, works with limited data
 - Challenge: Requires pre-trained models and fine-tuning expertise

Thanks

Kodanda Challa
